

PROGRAMMING · BEGINNER

JavaScript Foundations

Make the web respond to your ideas.



A free path from JavaScript fundamentals to browser applications and asynchronous code.

 6–8 weeks 4 modules 16 lessons[Explore the course →](#)

Concepts, worked examples, assignments and a coherent course project.

WHO IT IS FOR · WHAT YOU WILL LEARN

Your next capability.

Learn language fundamentals, DOM interaction, modules, asynchronous programming and modern browser development through practical mini-projects.



Write modern JavaScript



Build interactive browser interfaces



Use promises and async/await



Structure code with modules



Audience

- Web development beginners
- Designers moving into code
- Learners preparing for React or Node.js



Prerequisites

- Basic HTML and CSS are helpful

FROM CONCEPT TO EVIDENCE



One curriculum, three connected views

This guide summarizes the same modules and lessons shown on the course overview and in the learning workspace. The next pages explain the scope and practical evidence for every module.

DETAILED MODULE CURRICULUM · 1-2

Build the foundations.

The module names, lesson sequence and evidence below match the course learning workspace.

1 JavaScript Language Foundations

Build predictable transformations before connecting a user interface.

1.1 Values and variables

Use const when a binding will not be reassigned and let when reassignment is necessary.

1.2 Functions

A function should make its inputs and returned result clear.

1.3 Arrays and objects

Arrays represent ordered collections; objects represent named properties.

1.4 Errors and debugging

Read the error message and stack trace before changing code.

You will produce: Pure filtering function and input/output tests.

2 Browser and DOM

Connect a course filter to an accessible browser interface.

2.1 DOM selection

Select the specific DOM element needed and handle a missing match.

2.2 Events

An event listener responds to an interaction after the page has loaded.

2.3 Forms and validation

Browser validation improves feedback but does not secure a server.

2.4 Accessible UI state

Loading, empty, error and success states need meaningful text, not only a color change.

You will produce: Accessible filtering page and interaction checklist.

Practice is part of every module

Work through the example, record your reasoning, complete the module assignment and compare your work with the review criteria.

DETAILED MODULE CURRICULUM · 3-4

Apply, integrate and demonstrate.

The module names, lesson sequence and evidence below match the course learning workspace.

3 Asynchronous JavaScript

Represent asynchronous work without misleading loading or error states.

3.1 Promises

A promise represents an eventual result or rejection.

3.2 Async and await

await pauses the current async function, not the entire browser.

3.3 Fetch and JSON

fetch can resolve even when an HTTP response indicates an error.

3.4 Loading and error states

Rapid requests can finish out of order.

You will produce: Asynchronous page with explicit state transitions.

4 Modules and Capstone

Package the browser project so another developer can run and test it.

4.1 ES modules

ES modules declare explicit imports and exports.

4.2 Tooling basics

A development server provides a consistent origin and module loading.

4.3 Testing concepts

Tests compare an expected behavior with an observed result.

4.4 Interactive web app capstone

Deliver the filter application with clear states, accessible controls and reproducible setup.

You will produce: Interactive app, tests and reproducible README.

Practice is part of every module

Work through the example, record your reasoning, complete the module assignment and compare your work with the review criteria.

PROJECTS AND WORKING METHODS

Interactive browser application

Combine events, DOM updates and asynchronous behavior in a small web application that people can use and understand.

**Interactive interface****Documented event and data flow****Responsive project and test notes****Tools and methods**

Browser DevTools

VS Code

Git

Tools are learning choices, not partner endorsements.

Inside module 1: a worked example

```
const courses = ["C", "C++", "JavaScript"];
const query = "script";
const result = courses.filter(name =>
  name.toLowerCase().includes(query.toLowerCase()));
// result: ["JavaScript"]; courses remains unchanged.
```

A search result is a view of the original collection. A query matching nothing should not delete course records.

How your work is reviewed

Show the artifact, explain your decisions, test the stated assumptions and identify limitations. Module assignments build toward the course project rather than replacing it with a single example.

DELIVERY, COMPLETION AND NEXT STEPS

Know what you are signing up for.

**Delivery**

Self-paced online · 6–8 weeks

Confirm current schedule, cohort arrangements and enrollment terms with the academy.

**Completion requirements**

Complete the required coursework and module assignments, and pass the course assessment.

A course completion certificate is available after those requirements are met.

What the learning workspace includes**Follow the modules**

Expand the curriculum and open the exact lesson you need.

**Apply the concepts**

Read explanations, inspect examples and complete module practice.

**Keep your evidence**

Use resources, notes, discussion and assignments to support your learning.

Learning commitments, not outcome guarantees

Completion is not a placement, employment or performance guarantee. The course project is educational evidence; any production use requires appropriate review and validation.

Start with the curriculum. Continue with the first lesson.

Review your starting point, download this guide and explore the connected course experience.

[Go to the first lesson](#) →

Client-review edition. The static learning workspace saves demonstration progress locally; production enrollment, assessment, communications and certificate issuance are not connected.